Does CHAI plan better than CheckMate with the same operators?

Yes, on plan quality — and the gain comes from the control plane, not the extra operators.

Target MSP430FR5994 (no FPU, no SIMD, 10 KB RAM, 16 MHz) | 6 benchmarks x 3 repeats | 7B planner, temperature 0 | source `reports/plan_quality_data.json`

The comparison that matters

The obvious objection to CHAI is *"you just added ten more operators."* Arm **B** rules that out: it is CheckMate with the **identical 30-operator catalogue** CHAI uses, and no control plane. Arm **C** is CHAI with the same catalogue and the control plane on. Both arms choose from exactly the same menu, so any difference between them is the control plane.

	B CheckMate + 10 operators	C CHAI, same 30 operators
Control plane	off	**on**
Operator catalogue	T1-T30 (32 incl. auto-gen)	identical
Operators offered to the planner	32 (all)	**16 of 32** (median; 15-17)
Distinct per-function menus (ar-iclib)	1	**7**
Accepted plans	20	13
Hardware-impossible operators delivered	1	**0**
False operator labels the planner made (distinct IDs, 42 function attempts)	19	22
Unsupported labels *shipped* in the plan	**10**	**0**
Plans carrying at least one false label	**10 of 20 (50%)**	**0 of 13 (0%)**

1. Neither planner labels more accurately. Only one arm catches it.

Counting *distinct* false operator labels — one per (benchmark, seed, function, operator id) that ever failed conformance — over a denominator both arms record: **19** for arm B against **22** for arm C, in 42 function attempts each (0.45 versus 0.52 per attempt). Per-seed counts are B {5, 7, 7} and C {7, 7, 8}; the ranges overlap, so this run cannot separate the two planners, and if anything CHAI's model is marginally worse. What differs is what survives to the delivered artifact: **10 versus 0**. Exactly half of arm B's plans ship at least one operator label the code does not implement. CHAI detects the same mistakes and removes them before acceptance. Figure 1 overleaf shows both quantities side by side.

Why this number changed

An earlier version reported **35 versus 37** here and called it the same mislabelling rate. Both halves were wrong. `operator_conformance_violation_count` counts violation *events*, so one bad operator re-checked after each retry counts once per attempt — and the arms do not retry equally. Conformance is enforced only in the CHAI arms, which re-prompt after a failed check: 2.47 attempts per violation in arm C against 1.84 in arm B. The event count therefore penalised the arm doing the checking. There was also no denominator: 35 and 37 were bare totals. The corrected counts are recomputed by `scripts/operator_label_audit.py`.

Caveat — state this plainly when citing the figure

CHAI's zero is an *enforcement outcome*, not evidence of a better planner. CHAI prunes disproven labels before writing the accepted artifact, so the zero is true by construction. The honest claim is **"CHAI detects and removes the false labels that CheckMate ships"** — never "CHAI's model makes fewer mistakes." The 19-vs-22 bars are shown precisely so the figure cannot be misread.

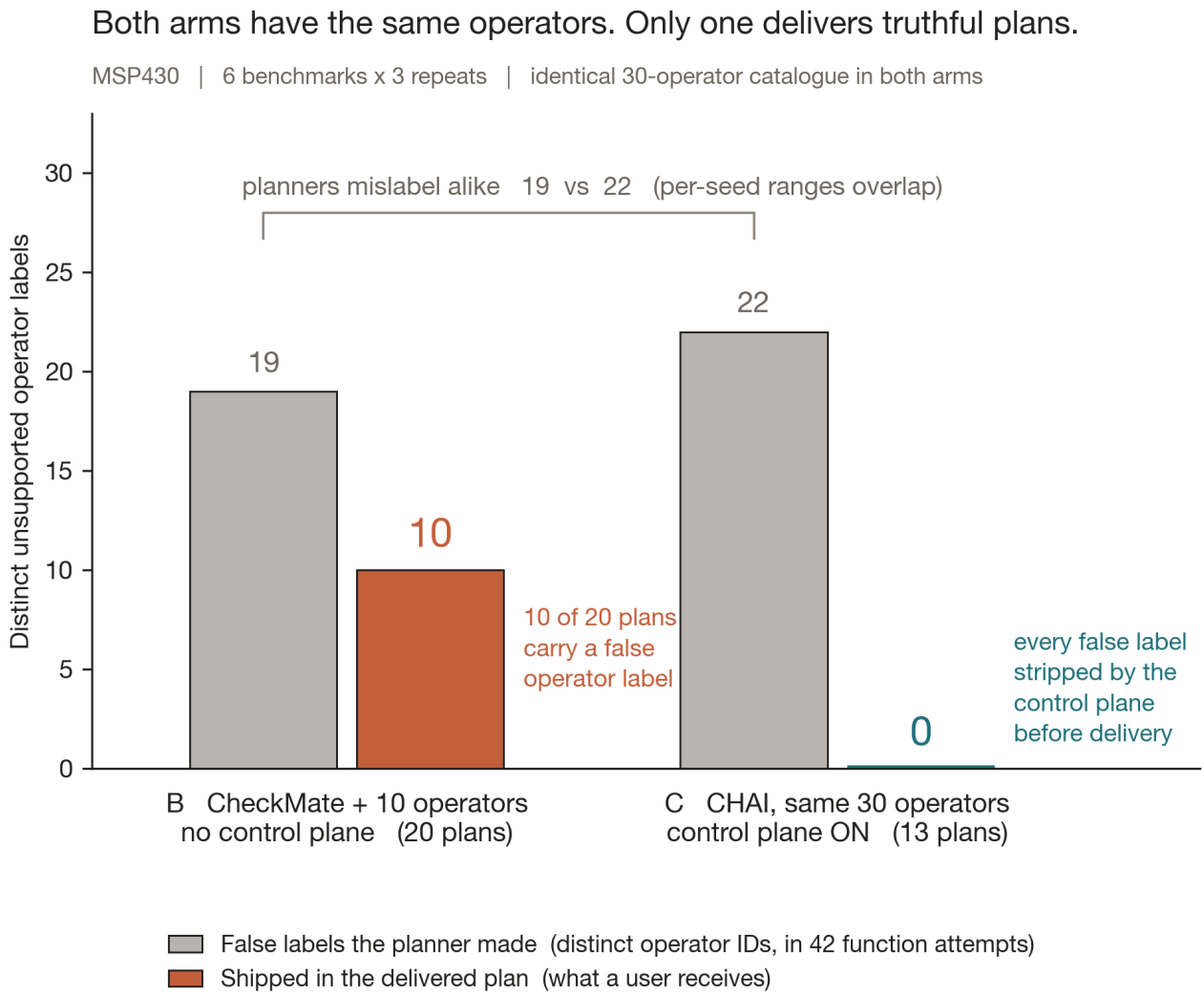


Figure 1. Grey = operator labels the planner got wrong during generation. Coloured = labels that survived into the delivered plan. Both arms draw from the identical 30-operator catalogue.

Where the control plane intervenes

2. The hardware gate removes half the menu before the model sees it.

CHAI filters the catalogue against the target's real capabilities: 32 operators down to a median of 16. Arm B sees all 32, including operators the chip physically cannot execute.

Concrete failure

On `fft-iclib`, arm B selected **T24 "Magnitude-Only FFT" — which requires an FPU — on a chip with no FPU.** The same plan also declared T24 without implementing it. One operator, two independent defects, shipped in a delivered plan. CHAI never offered T24 for this target.

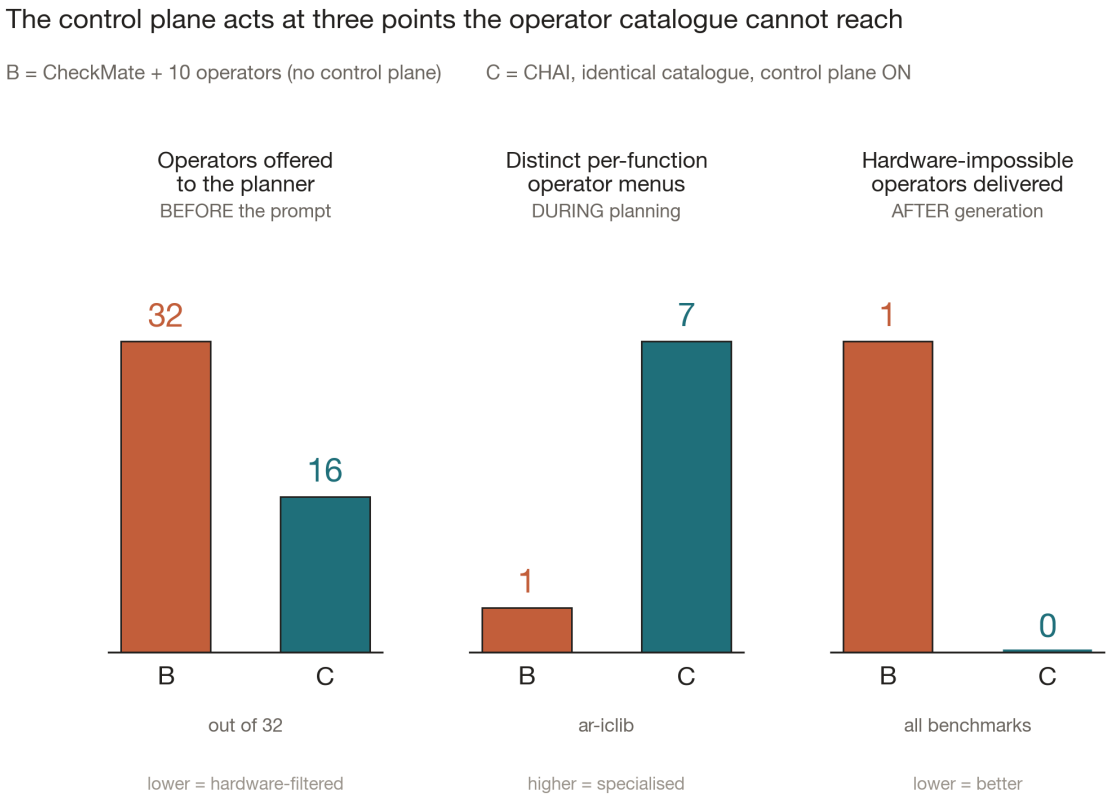


Figure 2. The three stages an operator catalogue alone cannot reach.

3. CHAI plans per function; CheckMate plans once.

On `ar-iclib`, arm B gave every function the *identical* operator menu (1 distinct menu, all three repeats). CHAI produced **7 distinct menus** across the same functions — each derived from what that function actually does.